ChatGPT

InBetween V1 — MVP Algorithm Specification for Problem–Solution Validation

Deliverables (Markdown + assets, ready for PDF conversion):
- Download the MVP spec (Markdown)
- Download `generate_priority_charts.py`
- Download `extract_focus_candidates.py` (pseudocode)
- Download `render_mermaid.sh`
- Download `lifecycle.mmd`
- Download `linking_flow.mmd`
- Pilot charts (PNG): Emma, Noah, Lina

Executive summary

This is an implementation-ready MVP spec for **InBetween V1**, designed to validate a single product hypothesis over a **4-week pilot**:

If students receive three clear, prioritised focus points after every private lesson (generated automatically by Lia from the lesson transcript) and can log practice + ask questions with minimal friction, then they will practise more consistently and progress faster between lessons—without creating extra admin work for coaches.

V1 is intentionally narrow:

- **Lia generates focus points from lesson audio → speech-to-text transcript** (not coach input).
- Students always see **exactly 3 actionable focus cards** (plus a passive "Other focuses" list).
- Coaches do **not** author focuses; they only validate:
- *potential duplicates / merge suggestions*,
- *low-confidence extraction edge cases*,
- and optionally *completion candidates*.

The user experience remains calm and minimal: the system should surface **decisions**, not analytics dashboards—aligned with key usability heuristics such as *aesthetic & minimalist design* and *recognition rather than recall*. [1]

MVP goals, constraints and scope

Goals for problem–solution validation

Product outcomes to validate: - **Structure**: students practise more consistently between lessons. - **Correctness**: they practise the right things (not only "comfortable" drills). - **Coach leverage**: coaches spend less time repeating and more time progressing.

Technical outcomes to validate: - **Transcript reliability** in real studios. - **Low-input robustness**: the app stays useful even if students skip logs. - **Low coach workload**: "coach actions" remain rare and fast.

Constraints and assumptions

Assumptions (pilot): - Private 1:1 lessons. - Coach records audio; Lia performs STT and extraction. - Student UI supports 3 focus cards.

Technical constraints: - Mobile retries happen; you need retry safety. HTTP defines idempotent methods and why repeating them after a network failure is safe in terms of intended server state. [2] - For retryable **POST** operations (practice logs, questions), implement **Idempotency-Key** semantics (store response for a key; replay on retry). Stripe's idempotency documentation is a practical reference implementation pattern. [3]

Privacy constraints (pragmatic MVP): - Audio/transcripts contain personal data; store minimally, restrict access to student + coach, define retention. - Store **evidence pointers** (short quotes + timestamps) rather than exposing full transcripts to coaches by default.

Minimal feature list for validation

Must-have features

Student: - Top-3 focus screen (3 cards, always). - "I practised" quick log. - Optional session log (duration optional). - Ask a question (typed), with question type. - Passive "Other focuses" list (no "work on this" CTA).

Coach: - Student list + "needs attention" strip (questions / duplicates / ready to validate). - "Potential duplicates" queue (merge / keep linked / unlink). - "Ready to validate" queue (validate completion or keep active).

Infrastructure / Lia: - Lesson audio upload + storage. - STT transcription (timestamps + confidence strongly preferred). - Utterance segmentation, best-effort diarisation (coach/student/unknown). - Focus candidate extraction + confidence gating + evidence pointers. - Focus link/cluster suggestions. - Priority Score Engine for selecting top 3.

Nice-to-have (only if time remains)

- Coach copy-edit focus wording (quick edit, no transcript work).
- Student one-tap "I'm not sure about this" (maps to confusion + coach attention).
- Push reminders (only if the pilot depends on them).

Transcript → focus extraction pipeline

The MVP extraction pipeline should prioritise **determinism, auditability, and confidence gating**.

Audio capture requirements

For better STT accuracy, capture should target: - **16 kHz+ sampling rate** when possible; lower sample rates can reduce accuracy, and re-sampling should generally be avoided. [4] - Prefer **lossless** encoding (e.g. LINEAR16/FLAC) where feasible; lossy codecs can reduce accuracy. [4] - Microphone placement close to speakers; avoid clipping; avoid heavy noise reduction/AGC when possible. [4]

MVP requirements: - Accept WAV (PCM) and M4A uploads. - Store metadata: format, sample_rate_hz, channels, duration_sec. - Upload to object storage using short-lived signed URLs.

STT options (MVP strategy)

Abstract via `transcripts.provider` so you can swap later.

Options that work well for MVP: - **OpenAI Whisper**: OpenAI describes Whisper as trained on 680,000 hours and robust to accents/noise and technical language. [5] - **Cloud STT** with diarisation: e.g. AWS Transcribe diarisation outputs speaker-labelled segments with timestamps. [6]

MVP recommendation: - Choose the path that yields timestamps + a confidence signal earliest.

Utterance segmentation and speaker labelling

Target utterance output:

Field	Meaning
utterance_id	stable id (`utt_42`)
speaker	coach / student / unknown
start_ms, end_ms	timestamps
text	utterance text
stt_confidence	0..1

Segmentation heuristics (starter): - split on pauses $\geq$ 700ms or punctuation + pause - cap utterance size ($\leq$ 12 seconds, $\leq$ 240 chars) - enforce one speaker per utterance if diarisation is available

Candidate extraction heuristics and phrase patterns

Each coach utterance is scored for pattern hits.

Pattern categories (starter): - **Repetition**: theme repeated $\geq$ 2× in window (use semantic similarity, not raw string match). - **Insistence / escalation**: “main issue”, “this is the problem”, “most important”, “you must”, “always”, “never”. - **Correction**: “no”, “don’t”, “stop”, “instead”, “again”, “try”. - **Drill prescription**: “do this drill”, “practice this”, “repeat this”, “five minutes”, “every day”.

Output per candidate: - title (short, action phrasing) - description (1–2 lines) - tier (critical/important/supporting) - initial importance_score + struggle_score - extraction_confidence (0..1) - evidence pointers (utterance refs + short quotes)

Confidence scoring and evidence pointers

Why confidence is mandatory: - STT systems can produce incorrect text, and model “hallucinations/confabulations” have been widely reported (including Whisper use cases). Even though dance coaching isn’t “medical”, this motivates confidence gating and evidence pointers for auditability. [7]

MVP confidence gating: - confidence < 0.55: do not create a focus; keep as low-confidence note - 0.55–0.74: create focus but mark needs_review=true - ≥ 0.75: create focus normally

Evidence pointers: - utterance_id, start_ms, end_ms, short quote (≤ 140 chars)

(Full pseudocode is provided in `extract_focus_candidates.py`.)

Priority Score Engine and lifecycle

Fields and what is persisted vs computed

Persist: - facts: title, tier, status, provenance, timestamps - metrics: importance, struggle, practice, coach_signal

Compute on read: - recency_score, inaction_penalty, priority_score

Priority formula

Starter formula (tunable):

$$priority = importance + struggle - (practice \times practiceWeight[tier]) + recency + tierModifier[tier] + coac$$

Clamp to [0..20].

Default parameters

These defaults match the month-long simulation values and are intentionally tunable:

Parameter	Default	Rationale
tierModifier[critical]	+3	keep critical visible
tierModifier[important]	+2	mid priority
tierModifier[supporting]	+1	low priority
practiceWeight[critical]	0.8	critical decays slower
practiceWeight[important]	1.0	neutral
practiceWeight[supporting]	1.2	supporting cools faster
recency day 0	+3	new lesson boost
recency day 1–3	+2	moderate
recency day 4–6	+1	small
recency ≥ day 7	0	none
clamp	0..20	avoid extremes

Recency and inaction rules

Recency table: - 0 days → 3 - 1–3 → 2 - 4–6 → 1 - ≥ 7 → 0

Inaction penalty (computed, low-input friendly): - graceDays = {critical: 4, important: 7, supporting: 999} - inactionDeltaPerWeek = {critical: 1.0, important: 0.5, supporting: 0.0} - inactionPenalty = inactionDeltaPerWeek[tier] × max(0, floor((days_since_practice − graceDays[tier]) / 7)) - cap at +3

Core pseudo-code

```
function compute_recency(days_since_mentioned):
  if days_since_mentioned == 0: return 3
  if 1 <= days_since_mentioned <= 3: return 2
  if 4 <= days_since_mentioned <= 6: return 1
  return 0

function compute_priority(focus, now):
  recency = compute_recency(days_since(now, focus.last_mentioned_at))
  tierMod = {critical:3, important:2, supporting:1}[focus.tier]
  w = {critical:0.8, important:1.0, supporting:1.2}[focus.tier]
  inaction = compute_inaction_penalty(focus, now)

  raw = focus.importance_score
      + focus.struggle_score
      - (focus.practice_score * w)
      + recency
      + tierMod
      + focus.coach_signal_score
      + inaction

  return clamp(raw, 0, 20)

function pick_top3(student_id, now):
  focuses = load_focuses(student_id, status="active")
  sort desc by compute_priority(focus, now)
  return first 3
```

Lifecycle states and transitions

States: - active - cooling_down - completed_candidate - validated_completed

Mermaid state diagram (Mermaid state diagram syntax documentation). [8]

```
stateDiagram-v2
  [*] --> active

  active --> cooling_down: priority<=4 for >=7d OR coach_signal<=-4 and
priority<=5
  cooling_down --> active: priority>=6 OR new_question OR coach_rementioned
```

```
  cooling_down --> completed_candidate: priority<=2 for >=14d AND practice>=6
AND no_confusion_14d
  completed_candidate --> validated_completed: coach_validates
  validated_completed --> active: coach_rementioned_in_lesson

  validated_completed --> [*]
```

Transition thresholds (starter): - active → cooling_down: priority ≤ 4 for ≥ 7 days OR coach_signal ≤ -4 and priority ≤ 5 - cooling_down → active: priority ≥ 6 OR new confusion question OR re-mention detected - cooling_down → completed_candidate: priority ≤ 2 for ≥ 14 days AND practice ≥ 6 AND no confusion questions for 14 days - completed_candidate → validated_completed: coach validates - validated_completed → active: re-mentioned again

Linking/clustering and coach validation

Definitions

- link_confidence ∈ [0,1]: probability two focuses share a theme
- merge_confidence ∈ [0,1]: probability they are duplicates (same focus)

Features for similarity

Starter features: - Semantic similarity using embeddings + cosine similarity (OpenAI recommends cosine similarity; embeddings are normalised, so cosine vs Euclidean give the same ranking). [9] - Lexical similarity (token overlap; normalised Levenshtein ratio). - Co-occurrence/reformulation evidence in later lessons. - Coach merge history.

Starter thresholds

Decision	Threshold
suggest link	link_confidence ≥ 0.75
auto-merge “clearly same”	merge_confidence ≥ 0.92 AND lexical_similarity ≥ 0.85
otherwise	coach review

Coach validation flow

Mermaid flowchart syntax documentation. [10]

```
flowchart TD
  A[New focus candidate created by Lia] --> B[Compute similarity vs existing
focuses (same student)]
  B --> C{link_confidence >= 0.75 ?}
  C -- no --> Z[No link suggested]
  C -- yes --> D[Create focus_link suggestion: potential_link]
  D --> E{merge_confidence >= 0.92<br/>AND lexical_similarity >= 0.85 ?}
  E -- yes --> F[Auto-merge (only "clearly same" cases)]
  E -- no --> G[Coach queue: "Potential duplicates"]
```

```
  G --> H{Coach action}
  H -- Merge --> F
  H -- Keep linked --> I[Same cluster, keep distinct focuses]
  H -- Unlink --> J[Reject link suggestion]
  K[Next lesson: co-occurrence evidence] --> L[Increase link_confidence
(+0.05, cap 1.0)]
  L --> G
```

Coach feedback, questions, and behaviour modulation

Coach feedback rules (detected from transcripts)

Category	Example phrases	Numeric effects
improved_strong	“much better”, “fixed”, “that’s good now”	coach_signal -= 4 (or -5 for “fixed”)
improved_moderate	“better”, “improving”, “getting there”	coach_signal -= 2
no_change	“still…”, “again…”, “same issue”	coach_signal += 0 (optional +1 if repeated)
escalation	“main issue”, “this is the problem”	coach_signal += 2 AND importance += 2, tier→critical possible

Question typology → struggle deltas

Type	Delta
confirmation	+0.5
clarification	+1.0
confusion	+2.0

Final delta: - struggle += delta × questionMultiplier(user)

User behaviour modulation (14-day window)

Compute: - Q = number of questions - S = number of practice sessions - L = number of quick logs - activity = S + 0.5×L - verbal_ratio = Q / (activity + 1)

Then: - silent if verbal_ratio < 0.3 → 1.2 - neutral if 0.3 ≤ verbal_ratio ≤ 2 → 1.0 - verbal if verbal_ratio > 2 → 0.7

This is deliberately light-weight and keeps the system useful under minimal input.

Event → effect mapping table

Starter deltas (tunable):

Event	Δimportance	Δstruggle	Δpractice	Δcoach_signal	Notes
FOCUS_CREATED_FROM_TRANSCRIPT	+base	+base	0	0	last_mentione
FOCUS_REMENTIONED_IN_LESSON	+2	0	0	0	last_mentione
COACH_ESCALATION_DETECTED	+2	0	0	+2	tier→critical p
COACH_IMPROVED_MODERATE_DETECTED	0	0	0	-2	
COACH_IMPROVED_STRONG_DETECTED	0	0	0	-4	
COACH_FIXED_DETECTED	0	0	0	-5	may trigger cooling_down
PRACTICE_QUICK_LOG	0	0	+1	0	last_practiced_
PRACTICE_SESSION_LOG	0	0	+2	0	last_practiced_
QUESTION_CONFIRMATION	0	+0.5×mult	0	0	last_question_
QUESTION_CLARIFICATION	0	+1.0×mult	0	0	last_question_
QUESTION_CONFUSION	0	+2.0×mult	0	0	last_question_
NOTE_NOT_SURE	0	+2.0×mult	0	0	also coach atte flag

Base values: - critical: importance 4, struggle 1 - important: importance 3, struggle 1 - supporting: importance 2, struggle 0

Minimal DB schema and SQL snippets

PostgreSQL JSON/JSONB functions & operators are documented in the official manual. [11]

Minimal schema

Table	Key fields	Purpose
users	id, email, role	auth
students	id, user_id, main_coach_id	linking
coaches	id, user_id	coach profile
lessons	id, student_id, coach_id, started_at, ended_at	lesson record
lesson_audio	id, lesson_id, storage_uri, duration_sec, format	audio artefact
transcripts	id, lesson_id, provider, language, raw_json(jsonb)	STT output
utterances	id, transcript_id, speaker, start_ms, end_ms, text, confidence	segmented transcript
focus_points	id, student_id, created_from_lesson_id, title, description, tier, status, cluster_id, alias_of	focuses
focus_metrics	focus_id, importance, struggle, practice, coach_signal, last_*	scores

Table	Key fields	Purpose
focus_evidence	id, focus_id, utterance_id, start_ms, end_ms, quote	evidence
focus_links	id, student_id, focus_a, focus_b, link_conf, merge_conf, lexical_sim, status	merge suggestions
practice_logs	id, student_id, focus_id, kind, duration_sec, idempotency_key, created_at	practice
questions	id, student_id, focus_id, type, text, idempotency_key, created_at	questions

Example SQL: top-3 selection

```
WITH scored AS (
  SELECT
    fp.id AS focus_id,
    fp.title,
    fp.tier,
    fp.status,
    GREATEST(0, LEAST(20,
      fm.importance
      + fm.struggle
      - (fm.practice * CASE fp.tier WHEN 'critical' THEN 0.8 WHEN
'important' THEN 1.0 ELSE 1.2 END)
      + (CASE
           WHEN DATE_PART('day', now() - fp.last_mentioned_at) = 0 THEN 3
           WHEN DATE_PART('day', now() - fp.last_mentioned_at) BETWEEN 1 AND
3 THEN 2
           WHEN DATE_PART('day', now() - fp.last_mentioned_at) BETWEEN 4 AND
6 THEN 1
           ELSE 0
         END)
      + (CASE fp.tier WHEN 'critical' THEN 3 WHEN 'important' THEN 2 ELSE 1
END)
      + fm.coach_signal
    )) AS priority_score
  FROM focus_points fp
  JOIN focus_metrics fm ON fm.focus_id = fp.id
  WHERE fp.student_id = $1
    AND fp.status = 'active'
    AND fp.alias_of IS NULL
)
SELECT *
FROM scored
ORDER BY priority_score DESC
LIMIT 3;
```

Example SQL: merging focuses

```
BEGIN;

UPDATE focus_points
SET alias_of = $1, status = 'validated_completed'
WHERE id = $2;

UPDATE practice_logs  SET focus_id = $1 WHERE focus_id = $2;
UPDATE questions      SET focus_id = $1 WHERE focus_id = $2;
UPDATE focus_evidence SET focus_id = $1 WHERE focus_id = $2;

UPDATE focus_metrics a
SET
  practice = a.practice + b.practice,
  importance = GREATEST(a.importance, b.importance),
  struggle   = GREATEST(a.struggle, b.struggle),
  coach_signal = a.coach_signal + b.coach_signal
FROM focus_metrics b
WHERE a.focus_id = $1 AND b.focus_id = $2;

DELETE FROM focus_metrics WHERE focus_id = $2;

COMMIT;
```

API endpoints and idempotency notes

HTTP idempotency semantics (RFC 9110). [12]
Idempotency keys (practical reference pattern). [13]

Recommended headers for retryable POSTs: - `Idempotency-Key: <uuid-v4>` - `Content-Type: application/json` - optional: `X-Client-Request-Id: <uuid-v4>`

Core endpoints (MVP): - POST /lessons (returns signed upload URL) - POST /lessons/{lessonId}/audio/complete - POST /lessons/{lessonId}/transcript (internal) - GET /students/{studentId}/focus/top?limit=3 - GET /students/{studentId}/focus/other - POST /students/{studentId}/practice_logs - POST /students/{studentId}/questions - GET /coaches/{coachId}/students - GET /coaches/{coachId}/students/{studentId}/merge_queue - POST /coaches/{coachId}/focus_links/{linkId}/resolve - POST /coaches/{coachId}/focus/{focusId}/validate_completed

Minimal UX notes

Student: - 3 focus cards (actionable) + passive “Other focuses” (no CTA). - Microcopy examples: - Header: “Focus for your next session” - Passive list: “Other focuses (not urgent)” - Empty state: “Your last lesson is still processing.”

Coach: - Home: minimal “needs attention” strip + student list. - Student detail: top-3 + questions + duplicates queue + ready-to-validate.

This keeps the interface aligned with minimalist usability principles. [1]

Pilot test plan, success metrics and simulation assets

Pilot duration: 4 weeks
Archetypes: Emma (disciplined), Noah (irregular/confused), Lina (high engagement/very verbal)

Success metrics (starter): - Median ≥ 2 active days per week - ≥ 2 practice logs/week per student - ≥ 60% of logs attached to top-3 focuses - Coach actions: median ≤ 3 merge reviews/student/week - Qualitative: “I know what to work on”; “I repeat less”.

Simulation timelines and charts are included in the Markdown file and PNG assets: - MVP spec Markdown - Emma chart, Noah chart, Lina chart

Risks, limitations and V1 → V2 roadmap

Key MVP risks: - Audio/transcript quality variance; capture best practices materially impact accuracy. [14] - Duplicate focus noise; mitigated via auto-link + coach merge validation. - Inactivity producing “same focuses forever”; mitigated by microcopy nudges, not hiding critical issues.

V1 → V2: - V1.5: stronger extraction classifier; better semantic clustering; better question classification. - V2: tuned/learned weights; improved diarisation; optional engagement layer (without undermining coach intent).

Instructions for PDF conversion and Mermaid rendering

Render Mermaid diagrams

Mermaid diagram syntax sources: state diagrams and flowcharts. [15]
Mermaid CLI installation/usage (official repo). [16]

1) Install CLI
`npm install -g @mermaid-js/mermaid-cli`

2) Render diagrams
`bash render_mermaid.sh`

Generate pilot charts

`python3 generate_priority_charts.py`

Convert Markdown to PDF (Pandoc + XeLaTeX)

Pandoc manual for PDF generation and `--pdf-engine`. [17]

```
pandoc inbetween_v1_mvp_spec_en.md \
  -o InBetween_V1_MVP_Spec.pdf \
  --pdf-engine=xelatex \
```

```
  --toc \
  -V geometry:margin=1in
```

If you want Mermaid diagrams embedded as images, render them first (`render_mermaid.sh`) and embed `lifecycle.png` / `linking_flow.png` inside the Markdown before running Pandoc.

Included scripts

- generate_priority_charts.py
- extract_focus_candidates.py
- render_mermaid.sh

---

1 https://media.nngroup.com/media/articles/attachments/Heuristic_Summary_compressed.pdf
https://media.nngroup.com/media/articles/attachments/Heuristic_Summary_compressed.pdf

2 12 https://datatracker.ietf.org/doc/html/rfc9110-
https://datatracker.ietf.org/doc/html/rfc9110-

3 13 https://docs.stripe.com/api/idempotent_requests?...=
https://docs.stripe.com/api/idempotent_requests?...=

4 14 https://docs.cloud.google.com/speech-to-text/docs/best-practices
https://docs.cloud.google.com/speech-to-text/docs/best-practices

5 https://openai.com/research/whisper
https://openai.com/research/whisper

6 https://docs.aws.amazon.com/transcribe/latest/dg/diarization.html
https://docs.aws.amazon.com/transcribe/latest/dg/diarization.html

7 https://www.wired.com/story/hospitals-ai-transcription-tools-hallucination
https://www.wired.com/story/hospitals-ai-transcription-tools-hallucination

8 15 https://mermaid.js.org/syntax/stateDiagram.html
https://mermaid.js.org/syntax/stateDiagram.html

9 https://help.openai.com/en/articles/8984345-which-distance-function-should-i-use
https://help.openai.com/en/articles/8984345-which-distance-function-should-i-use

10 https://mermaid.js.org/syntax/flowchart.html
https://mermaid.js.org/syntax/flowchart.html

11 https://www.postgresql.org/docs/current/functions-json.html
https://www.postgresql.org/docs/current/functions-json.html

16 https://github.com/mermaid-js/mermaid-cli
https://github.com/mermaid-js/mermaid-cli

17 https://pandoc.org/MANUAL.html?from=20423&from_column=20423
https://pandoc.org/MANUAL.html?from=20423&from_column=20423